

Guide de Sauvegarde et Restauration — La Termitière

Version : 1.0

Statut : Production

Cible : Équipe technique / DevOps

Dernière mise à jour : Juillet 2026

1. Objectif

Ce document décrit comment effectuer des sauvegardes régulières de la base de données PostgreSQL gérée par Supabase, et comment restaurer ces données en cas de crash majeur du VPS.

2. Sauvegarde (Backup)

Puisque nous utilisons Supabase en mode *Self-Hosted* via Docker, la base de données tourne dans le conteneur `supabase-db`.

2.1 Préparation du dossier de sauvegarde

Avant toute chose, il faut créer le dossier qui va accueillir les sauvegardes (sinon les commandes échoueront silencieusement) :

```
mkdir -p /home/bawa/backups
```

2.2 Sauvegarde manuelle complète (`pg_dumpall`)

C'est la méthode recommandée pour faire un "snapshot" complet de toute l'instance PostgreSQL (incluant les rôles, les schémas `auth`, `public`, `storage`, etc.). On utilise `gzip` pour compresser le fichier et économiser énormément d'espace disque.

```
# Lancer la sauvegarde et la compresser directement
docker exec supabase-db pg_dumpall -c -U postgres | \
  gzip > /home/bawa/backups/backup_termitiere_$(date +%Y%m%d_%H%M%S).sql.gz

# Vérifier que le fichier a bien été créé et n'est pas vide
ls -lh /home/bawa/backups/
```

⚠ Note technique : Ne mets jamais l'option `-t` dans `docker exec` quand tu rediriges la sortie vers un fichier (`>`). Cela corrompt le fichier SQL avec des retours à la ligne Windows (`\r\n`).

2.3 Sauvegarde ciblée des données métier (schéma `public`)

Si vous ne voulez sauvegarder que vos tables métier (`tp_*`) :

```
docker exec supabase-db pg_dump -U postgres -d postgres -n public | \
  gzip > /home/bawa/backups/backup_public_$(date +%Y%m%d).sql.gz
```

2.4 Automatisation (Cron Job)

Pour ne pas oublier, il faut automatiser la sauvegarde (tous les jours à 3h du matin). Comme la commande est longue, la bonne pratique est de créer un petit script.

Étape 1 : Créer le script de sauvegarde

```
nano /home/bawa/backups/backup.sh
```

Collez ce contenu dedans :

```
#!/bin/bash
# 1. Générer et compresser la sauvegarde
docker exec supabase-db pg_dumpall -c -U postgres | \
  gzip > /home/bawa/backups/backup_$(date +%Y%m%d).sql.gz

# 2. Nettoyer (supprimer les backups de plus de 7 jours)
find /home/bawa/backups/ -type f -name "*.sql.gz" -mtime +7 -delete
```

(Sauvegardez : `Ctrl+X` puis `Y` puis `Entrée`)

Étape 2 : Rendre le script exécutable

```
chmod +x /home/bawa/backups/backup.sh
```

Étape 3 : L'ajouter aux tâches planifiées (Cron)

```
crontab -e
```

(Si on te demande de choisir un éditeur, tape "1" pour nano).

Ajoute cette ligne très courte tout en bas du fichier :

```
0 3 * * * /home/bawa/backups/backup.sh
```

Sauvegarder et quitter l'éditeur : `Ctrl+X`, puis `Y`, puis `Entrée`.

⚠ ATTENTION : Ces backups sont sur le même VPS. Si le disque dur du VPS lâche, vous perdez tout. Il est crucial d'utiliser `rsync` ou `scp` pour copier ces fichiers `.sql.gz` sur un autre ordinateur ou un stockage cloud (S3) externe.

3. Restauration (Recovery)

En cas de problème (données effacées par erreur, ou migration ratée), voici comment restaurer un backup `.sql`.

3.1 Procédure de restauration complète

- Couper les accès au backend pour éviter des écritures pendant la restauration :

```
cd /home/bawa/supabase/docker
docker compose stop rest auth realtime
```

- Injecter le fichier `.sql.gz` dans la base de données :

```
# Remplace "20260704_120000" par la date exacte de ton fichier
zcat /home/bawa/backups/backup_termitiere_20260704_120000.sql.gz | \
  docker exec -i supabase-db psql -U postgres -d postgres
```

- Redémarrer les services :

```
docker compose start rest auth realtime
```

- Tester le bon fonctionnement via PostgREST (comme vu dans le guide de dépannage).